

Program Analysis

Learning Goals Today

- Be able to explain how soundness and completeness trade off in the design of tools that aim to find bugs automatically.

Spot the Bug

```
1. static OSStatus
2. SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa,
3.                                   SSLBuffer signedParams,
4.                                   uint8_t *signature,
5.                                   UInt16 signatureLen) {
6.     OSStatus err;
7.     ...
8.     if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
9.         goto fail;
10.    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
11.        goto fail;
12.    goto fail;
13.    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
14.        goto fail;
15.    ...
16. fail:
17.    SSLFreeBuffer(&signedHashes);
18.    SSLFreeBuffer(&hashCtx);
19.    return err;
20.}
```

Spot the Bug

```
1. static OSStatus
2. SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa,
3.                                   SSLBuffer signedParams,
4.                                   uint8_t *signature,
5.                                   UInt16 signatureLen) {
6.     OSStatus err;
7.     ...
8.     if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
9.         goto fail;
10.    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
11.        goto fail;
12.        goto fail;
13.    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
14.        goto fail;
15.    ...
16. fail:
17.    SSLFreeBuffer(&signedHashes);
18.    SSLFreeBuffer(&hashCtx);
19.    return err;
20.}
```

KEVIN POULSEN

SECURITY FEB 22, 2014 11:27 AM

Behind iPhone's Critical Security Bug, a Single Bad 'Goto'

Like everything else on the iPhone, the critical crypto flaw announced in iOS 7 yesterday turns out to be a study in simplicity and elegant design: a single spurious "goto" in one part of Apple's authentication code that accidentally bypasses the rest of it.



tomorrow
belongs to those who embrace it
today



trendingtechinnovationbusinesssecurityadvicebuying

[/ tech](#)

Home / Tech / Security

Apple's 'goto fail' tells us nothing good about Cupertino's software delivery process

The fact that Apple's infamous SSL validation bug actually got out into the real world is pretty terrifying.



Written by **Matt Baxter-Reynolds**, Contributor
March 19, 2014 at 3:00 a.m. PT

How Should Apple Have Found the Bug?

- Better code review?
- Better testing?
- Formal verification?
- Today's approach: *analyze the program's source code*

Spot the Bug

```
1. static OSStatus
2. SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa,
3.                                  SSLBuffer signedParams,
4.                                  uint8_t *signature,
5.                                  UInt16 signatureLen) {
6.     OSStatus err;
7.     ...
8.     if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
9.         goto fail;
10.    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
11.        goto fail;
12.    goto fail;
13.    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
14.        goto fail;
15.    ...
16. fail:
17.    SSLFreeBuffer(&signedHashes);
18.    SSLFreeBuffer(&hashCtx);
19.    return err;
20.}
```



This code is unreachable.
Isn't that a warning sign?

Hard-To-Find Bugs

- Often on a hard-to-execute codepath (need specific test cases)
- Can't actually test code exhaustively (too many paths, way too many states)
- Instead:
 - Identify relevant properties (e.g. code never dereferences NULL)
 - Try to prove program has those properties

Program Analysis

- Goal: answer questions about a program
- Examples:
 - Might this code ever dereference NULL?
 - Can I find any cases in which this code definitely divides by zero?

Soundness and Completeness

- A **sound** analysis finds all bugs (in a category of bugs).
 - No false negatives (doesn't fail to find a bug)
- A **complete** analysis only reports bugs (in a category of bugs).
 - No false positives (doesn't report bogus bugs)
- Generally, analyses are either **unsound** or **incomplete** (or both!)

Trust

- If a sound analysis says a program is safe, it is
 - (won't miss bugs)
- If a complete analysis reports a bug, the program is buggy
 - (won't report bogus bugs)

Static Analysis vs. Dynamic Analysis

- **Static analysis** is the analysis of programs *without* executing them
 - Usually want to find bugs or prove safety properties (the absence of bugs)
 - Often, static analyses can be made sound
- **Dynamic analysis** allows running programs
 - Dynamic analyses are more likely to be complete (only report bugs)

Static Analysis

- Key properties:
 - Liveness: "this good thing eventually happens" (e.g. server generates a response)
 - Safety: "this bad thing never happens" (e.g. dividing by zero)

Example

```
def n2s(n: int, b: int):  
    if n <= 0: return '0'  
    r = ''  
    while n > 0:  
        u = n % b  
        if u >= 10:  
            u = chr(ord('A') + u-10)  
        n = n // b  
        r = str(u) + r  
    return r
```

- What types can 'u' have at each line?
- Can 'u' be negative?
- Will **n2s** always return a value?
- Can there be division by zero?
- Will the returned value ever include a '-'?

Static Analysis Techniques

- Linters
 - Shallow syntax analysis (unsound, incomplete, unclear properties)
- Type checking (lots of research here)
 - Ensures program has well-defined semantics
- Data flow analysis, abstract interpretation (lots of research here too)
 - Is `a[i]` always within bounds?
 - Typical answers: "yes", "no", "maybe"

Rice's Theorem (Henry Rice, 1953)

- "Any nontrivial property about the language recognized by a Turing machine is undecidable."
- Implication: interesting static analyses will be imperfect (some false positives, false negatives, or sometimes not terminate)

Proof Sketch (by Contradiction)

- Suppose you have a function, **divides_by_zero**, that determines whether an input program divides by zero.

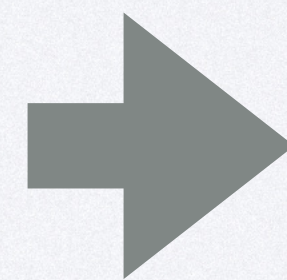
```
int oops(program p, input i) {  
    p(i);  
    return 5/0;  
}
```

```
bool halts(program p, input i) {  
    return divides_by_zero(oops(p,i));  
}
```

Pattern-Based Bug Detection

- e.g. SpotBugs
- Example: if a method acquires a lock, it should release it on all paths

```
Lock l = ...;  
l.lock();  
try {  
    // do something  
    l.unlock();  
}
```



```
Lock l = ...;  
l.lock();  
try {  
    // do something  
} finally {  
    l.unlock();  
}
```

Oops! `l` remains locked if an exception is thrown

Tradeoffs

- Analysis must be super fast
- In general, these pattern-based detectors are **unsound** and **incomplete**
- Google recommends static analyzers have $< 10\%$ false positives [Sadowski]
 - Otherwise developers will turn them off!

Type-Based Approaches

- Idea: Extend the type system to enable reasoning about important properties

```
public class NullnessExample {  
    public static void main(String[] args) {  
        Object myObject = null;  
        System.out.println(myObject.toString());  
    }  
}
```

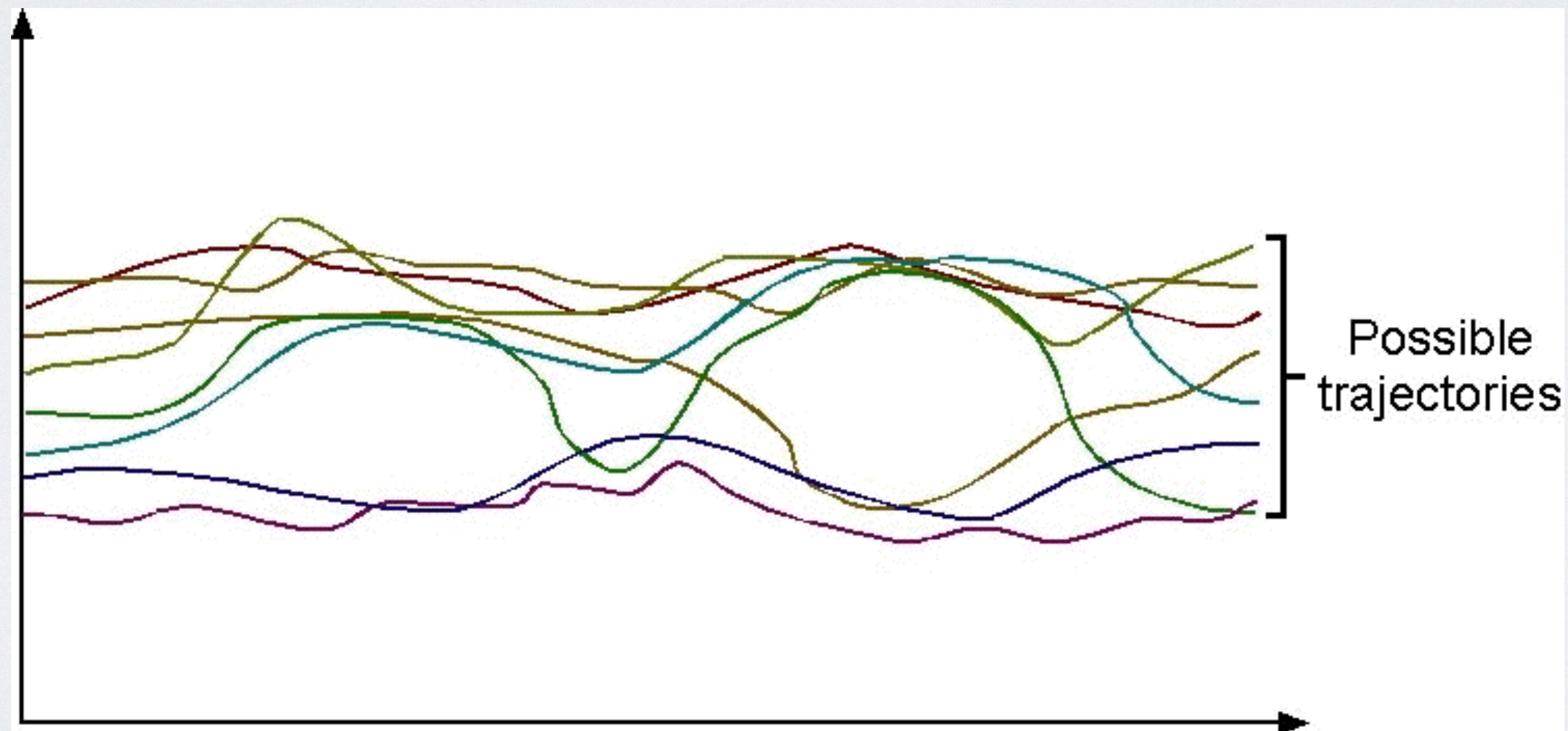
```
$ javacheck -processor org.checkerframework.checker.nullness.NullnessChecker NullnessExample.java
```

```
NullnessExample.java:9: error: [dereference.of.nullable] dereference of possibly-null reference myObject  
        System.out.println(myObject.toString());  
                           ^
```

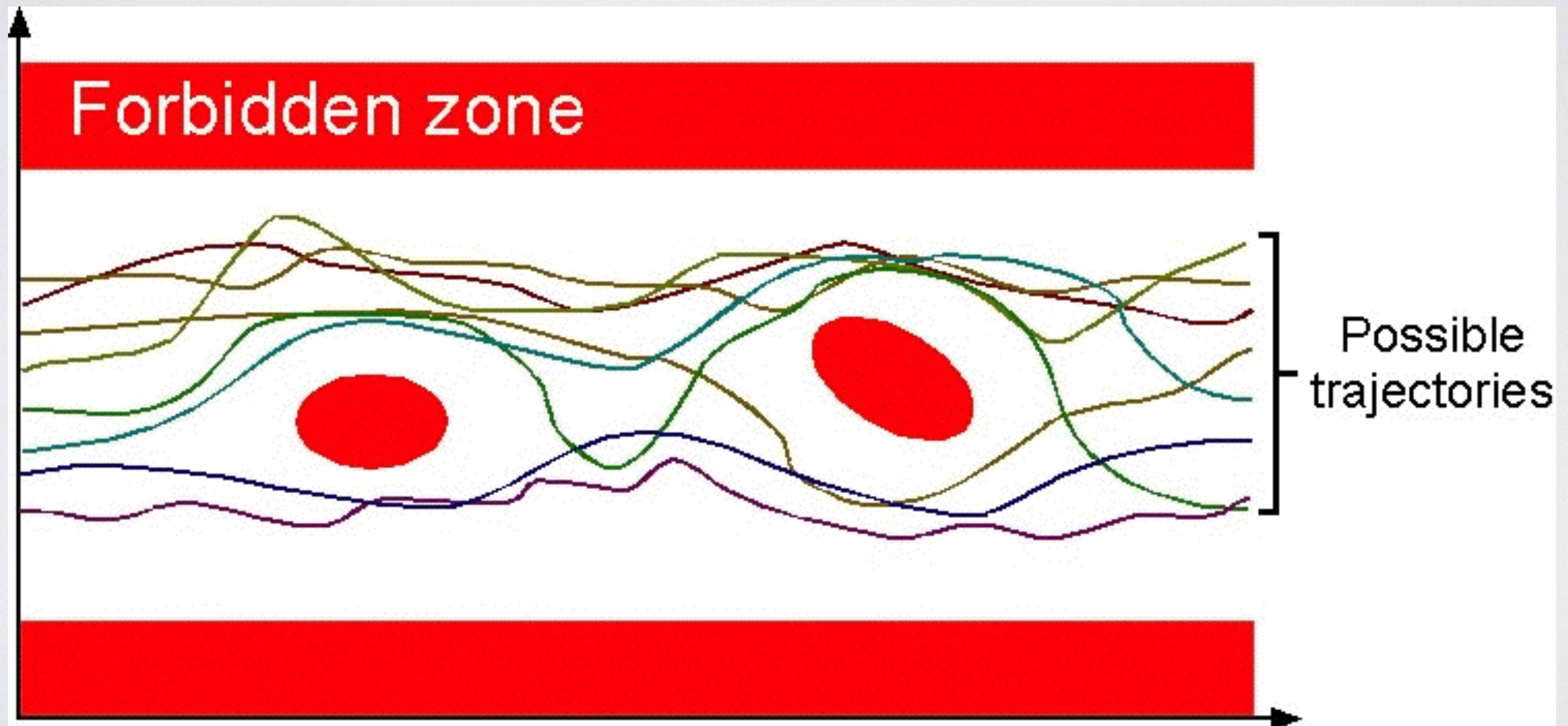
1 error

Abstract Interpretation

- Concrete semantics: all possible executions of a program

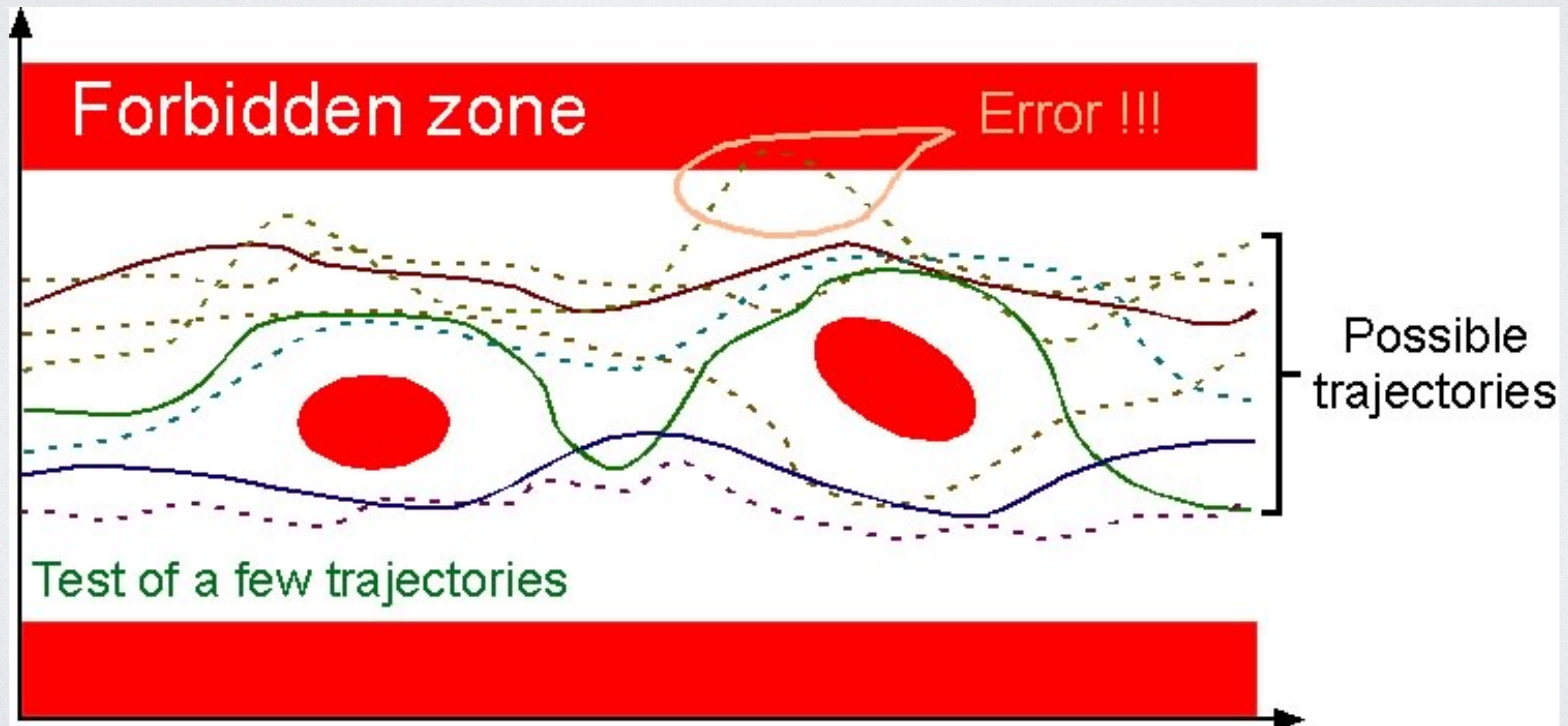


Safety Properties



Testing

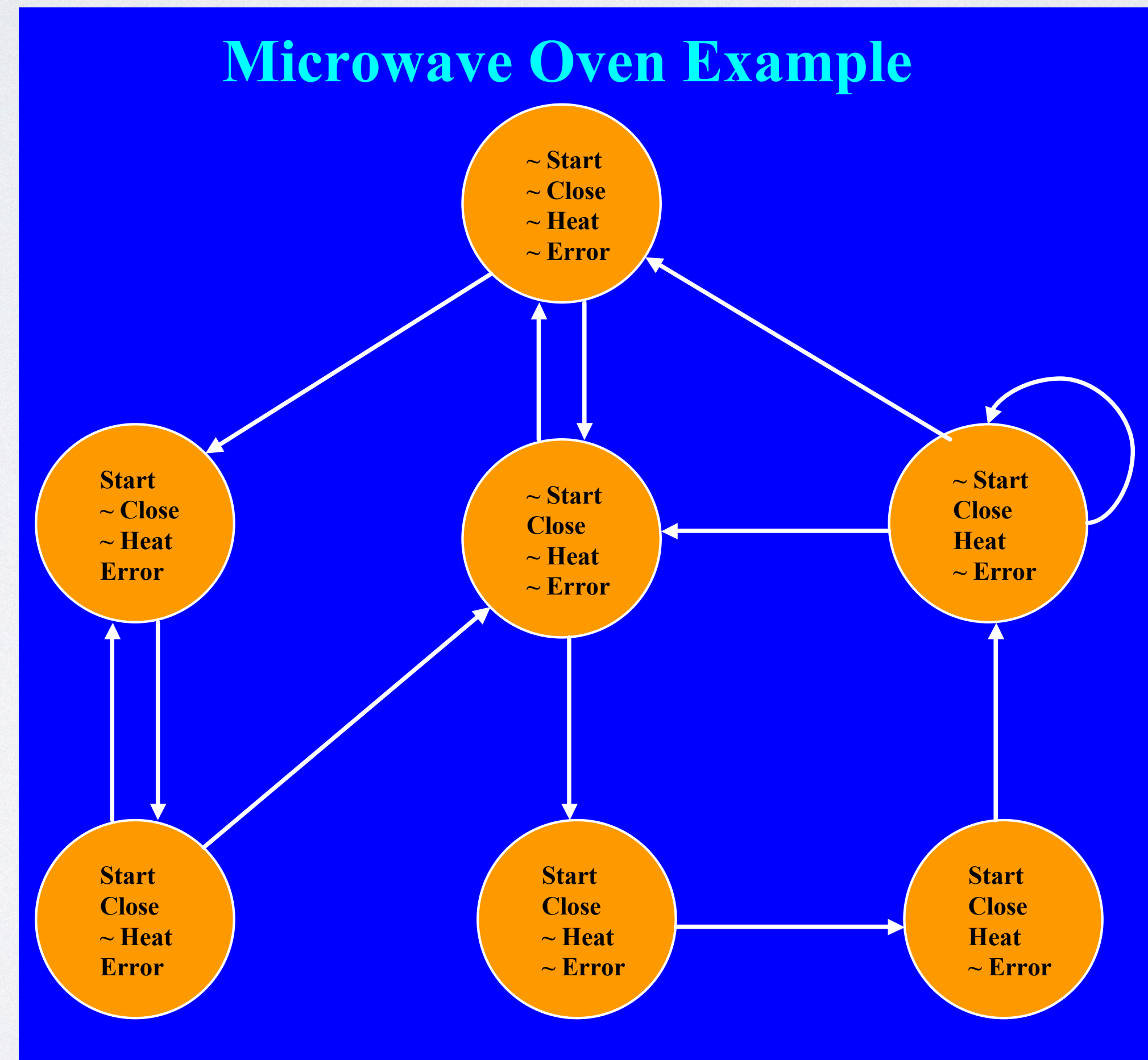
- Can only test some of the possible trajectories



Model Checking

- Goal: Explore all possible execution paths (via logic)
- Problem: too many execution paths (loops, recursion)
- Approach: *bounded* model checking (execute loops at most N times)

Model Checking Example (Ed Clarke)



Microwave Specification (Clarke)

- The oven doesn't heat up until the door is closed.
- Not heat_up holds until door_closed
 - $(\sim \text{heat_up}) \text{ U door_closed}$

Model Checking Formalization (Clarke)

- Let M be a state-transition graph.
- Let f be the specification in temporal logic.
- Find all states s of M such that $M, s \models f$.

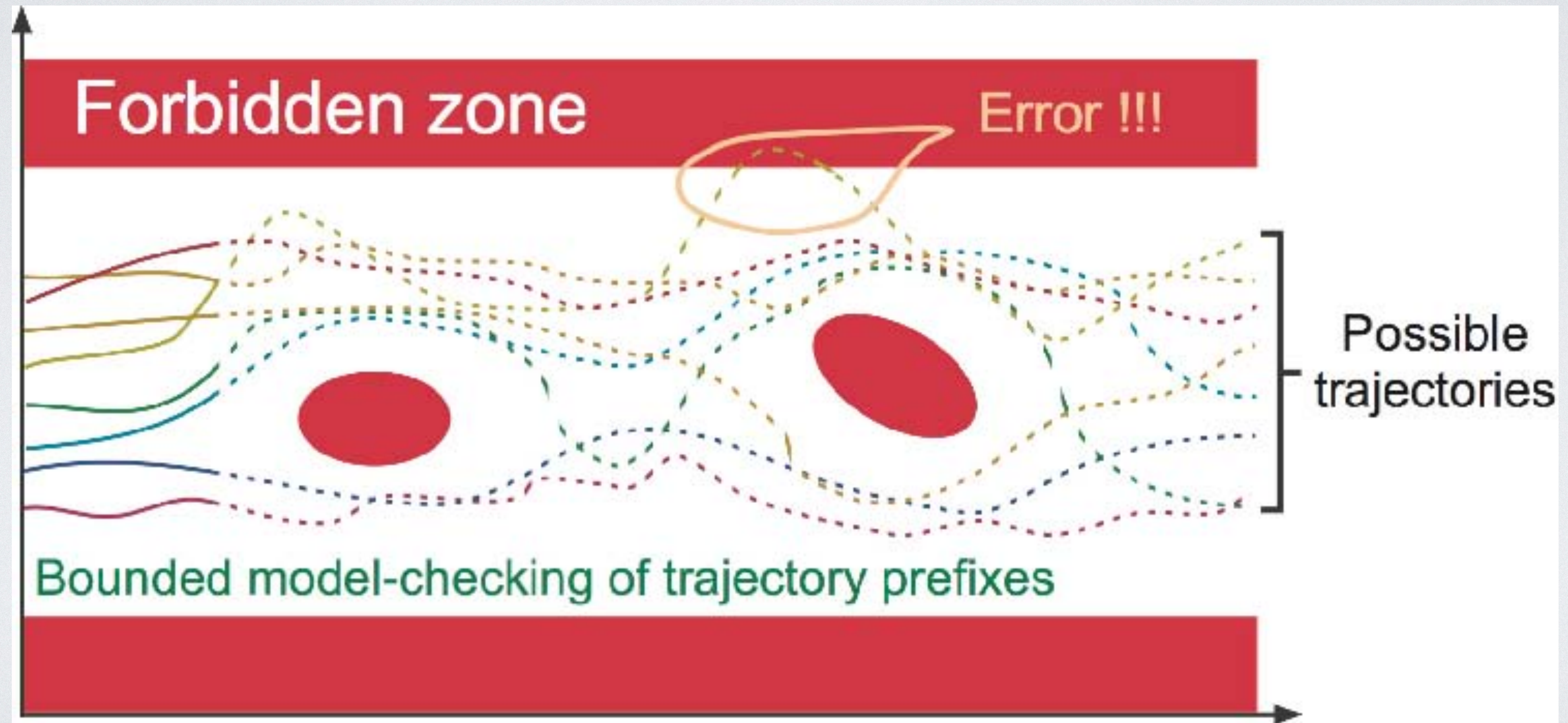
Tradeoffs

- Advantages: don't have to write proofs
- Disadvantages: state explosion; have to formally specify desired properties

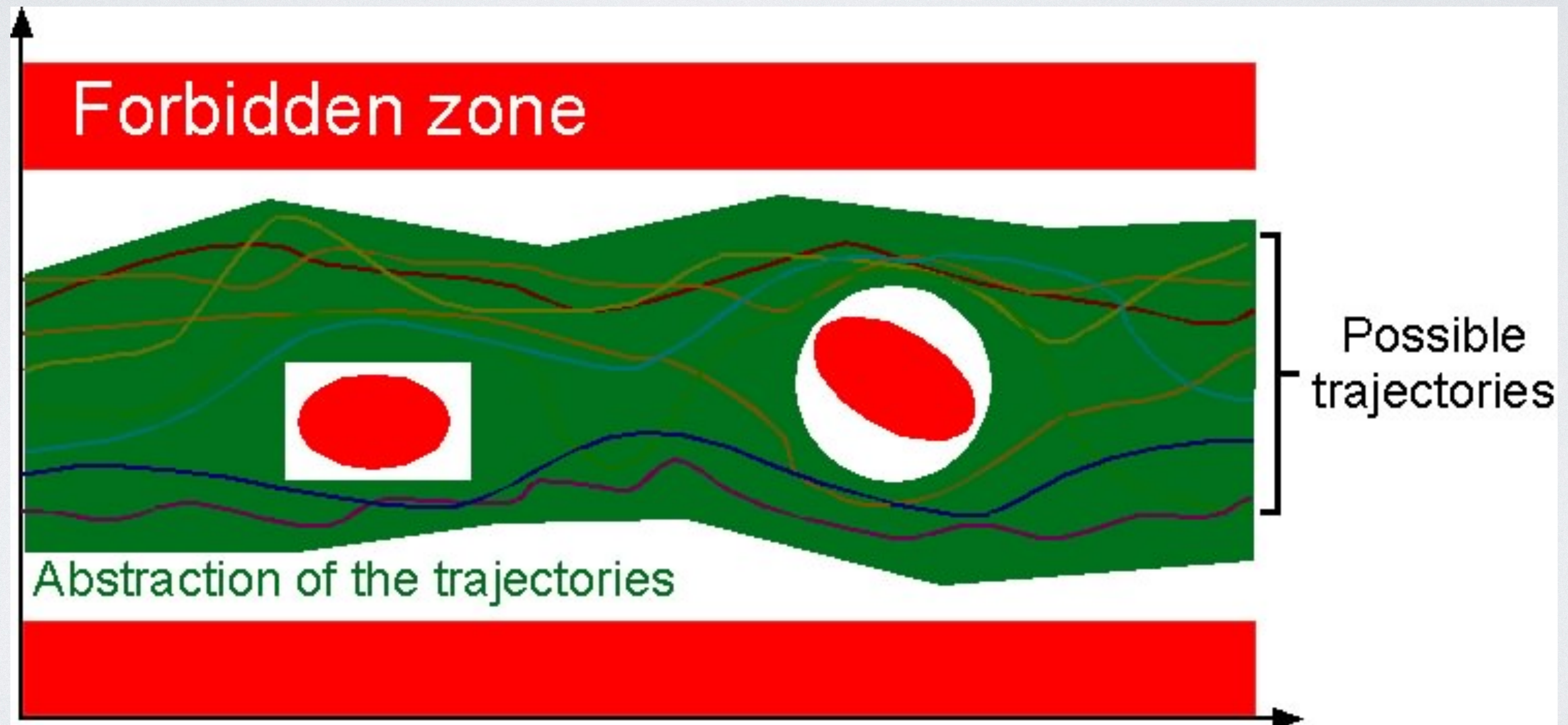
Model Checking Success Story

- In early 2000s: Windows users were plagued by blue screens of death
- Most common cause: driver bugs (not Microsoft's fault)
- Solution: model check drivers

Bounded Model Checking



Abstract Interpretation

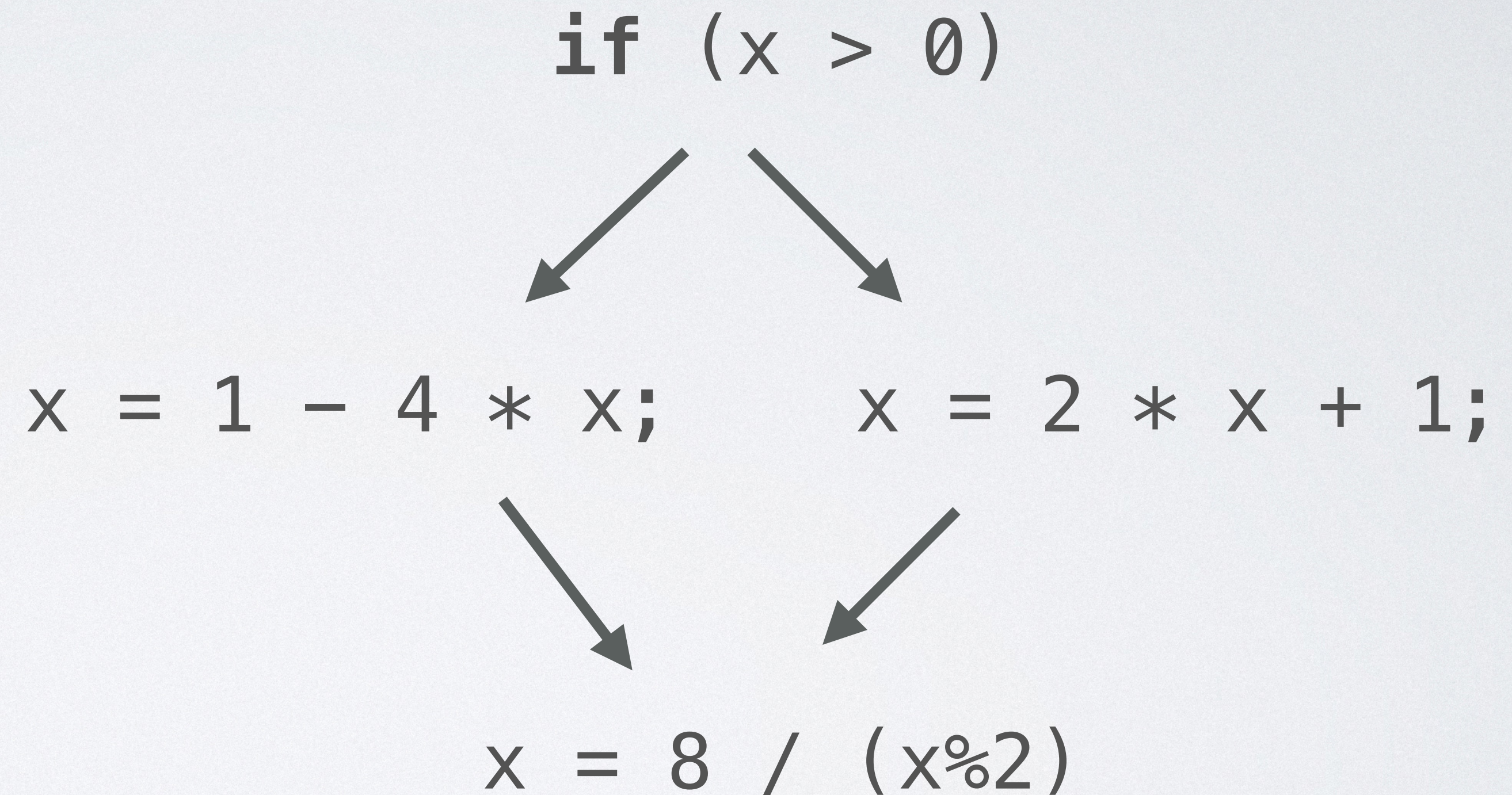


Example: Numerical Intervals

- Ideally: figure out what values variables can have
 - But that requires running the program with all inputs 😞
- Instead, track bounds $[L, H]$ for each variable

Will This Code Divide by Zero?

```
if (x > 0) {  
    x = 2 * x + 1;  
}  
else {  
    x = 1 - 4 * x;  
}  
  
x = 8 / (x%2)
```



Defining an Abstract Domain

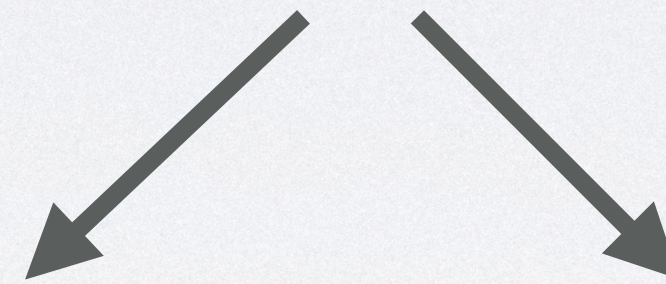
- We need to know if $(x \% 2)$ could be 0
- Let's track whether x could be even or odd.
 - Don't track all the values x could have.
- Abstract domain: {even, odd}

Analysis

$\{-\infty, \infty\}; \{\text{even}, \text{odd}\}$

if ($x > 0$)

$\{-\infty, 0\}; \{\text{even}, \text{odd}\}$

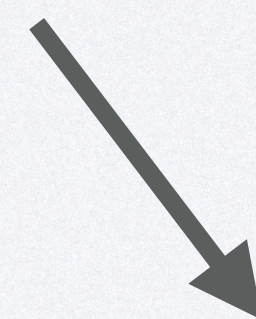


$\{1, \infty\}; \{\text{even}, \text{odd}\}$

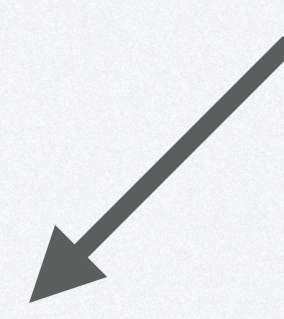
$x = 1 - 4 * x;$

$x = 2 * x + 1;$

$\{1, \infty\}; \{\text{odd}\}$



$\{3, \infty\}; \{\text{odd}\}$



$\{1, \infty\}; \{\text{odd}\}$

$x = 8 / (x \% 2)$

Activity

- Compare abstract interpretation with linters. In what contexts might you prefer each approach?
- (submit on Gradescope)

Conclusion

- We can find lots of bugs by analyzing code
- But analyses are generally unsound, incomplete, or both
- Software engineers hate false positives, so choose analyses wisely